4.3 Brief Control Instruction

This section aims to guide you on how to obtain the action data of MiniArm via the Arduino platform, as well as how to write and execute the action.

1. Read Action

1.1 Basic Program Download

The source code of the program is located in: “1.Tutorials\4.Secondary Development Basics and Environment Configuration\4.3 Brief Control Introduction\read_mini\read_mini.ino”.

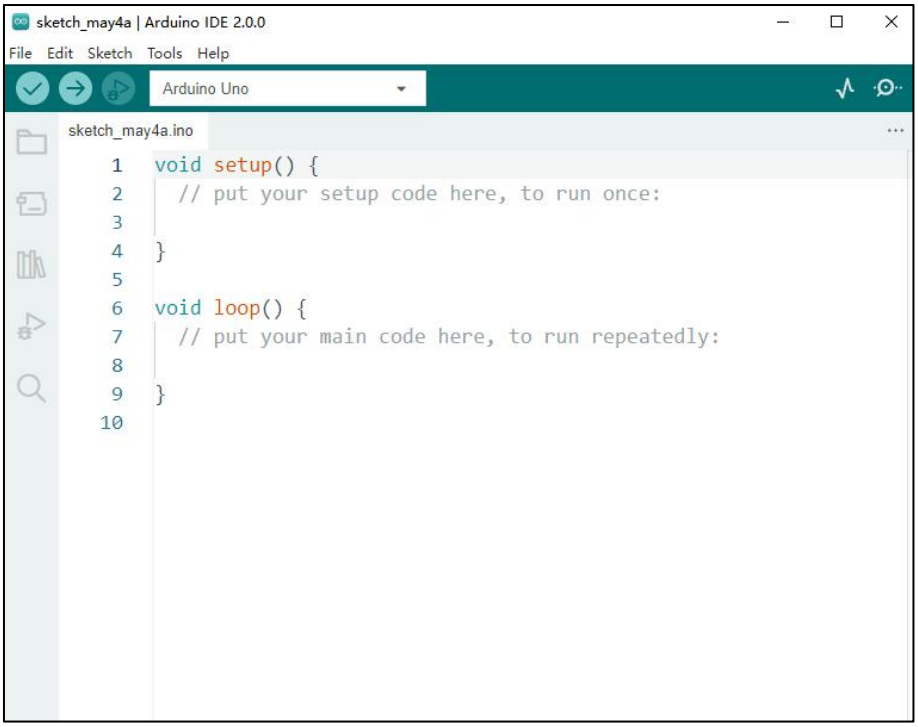
Note:

◆ Remove the Bluetooth module before downloading the program. Or the program will fail to download because of the serial port conflict.

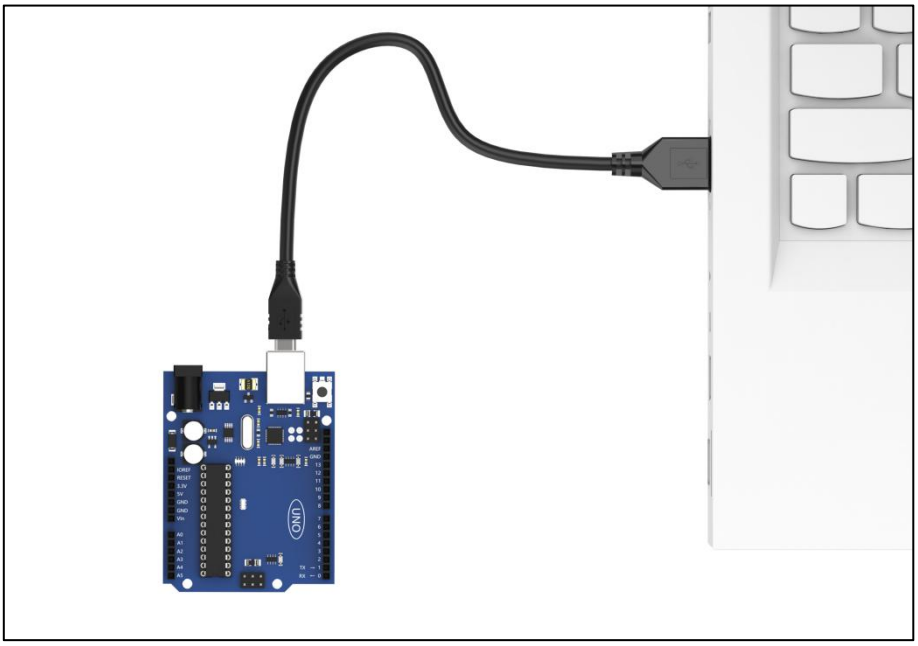
◆ Please switch the battery box to “OFF” when connecting the Type-B download cable. This action prevents the download cable from accidentally touching the power pins on the expansion board, which may cause a short circuit.

---

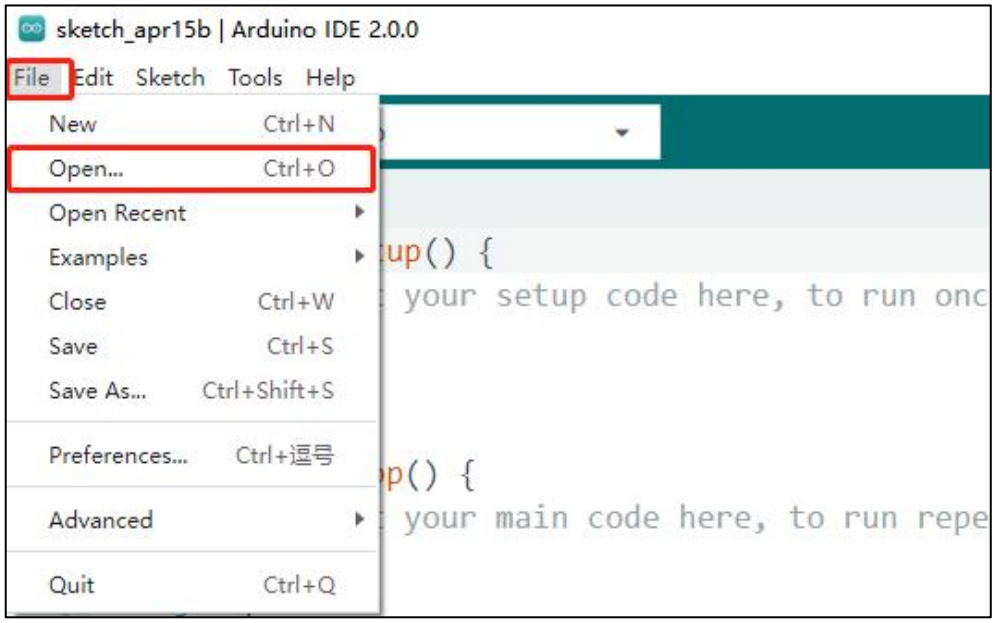
1) Double-click to open it.



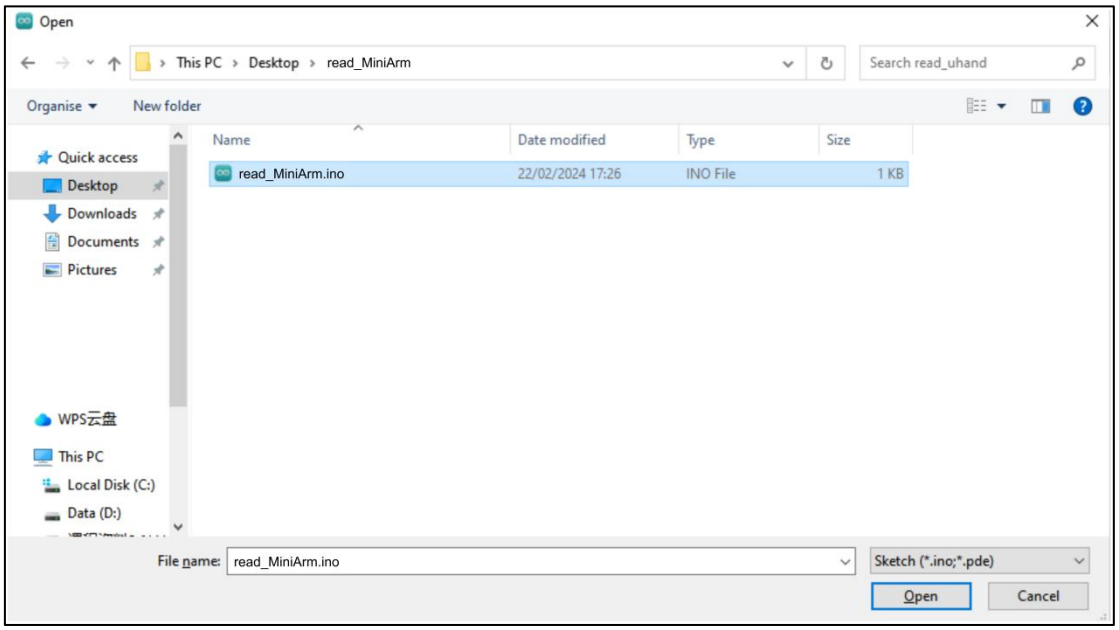
2) Connect the Arduino to the computer with the UNO cable (Type-B).



3) Click “File -> Open”.

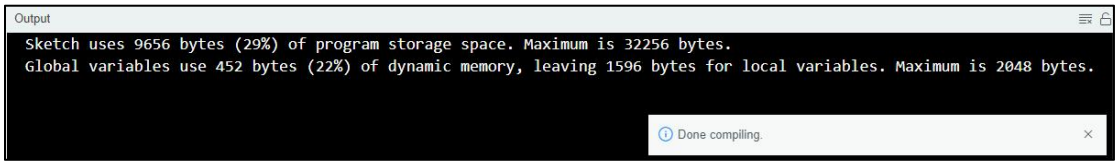


4) Access “1.Tutorials\4.Secondary Development Basics and Environment Configuration\4.3 Brief Control Introduction”. Select the “read_mini.ino” program in the folder “read_mini” and click “Open”.

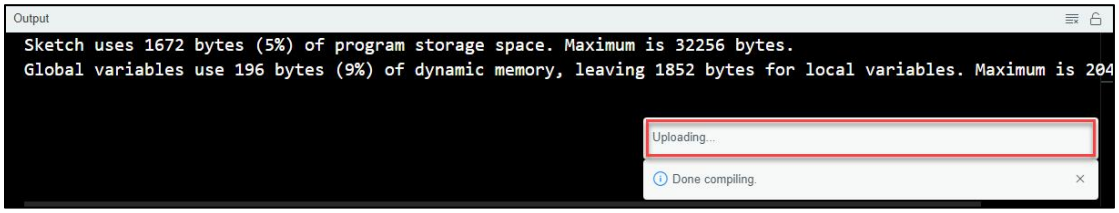


5) Locate the Arduino development board in “Select Development Board”. (Take Arduino Uno as an example. COM ports are not unique, and their numbers can be checked in the computer's device management. Take COM6 as an example.)

6) After confirming that everything is correct, click to verify the program. If the program is correct, the status area shows “Compiling Project -> Completed” in turn. Then it displays the number of bytes used by the current project, the program storage space, etc.



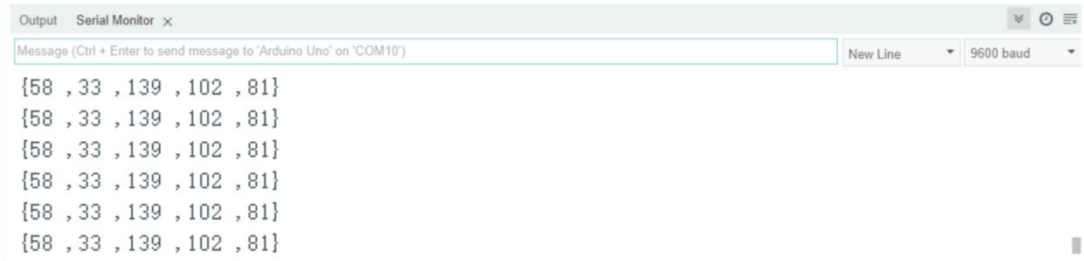
7) After successful compilation, click to upload the program to the development board. The status area shows “Compiling Project-> Uploading-> Upload successfully” in turn. After that, the status area stops printing and uploading information.



1.2 Get Action Data

1) Please keep the Type-B connected to the computer. (Do not plug off the cable during the process to ensure communication.)

2) Press “Ctrl+Shift+M” to open the serial port monitor to check real-time angle data of servos No.1 to No.5.



3) By rotating the knobs (S1 to S5) on MiniArm, you can adjust the corresponding angles of servos. And then save the action data.

2. Action Running

2.1 Write Action

1) Open the program file “mini_action\ mini_action.ino” in the same path as this section.

2) Locate the code highlighted in the image below, and write the obtained action data into it.

```
static int8_t servo_offset[6] = { 0 , 0 , 0 , 0 , 0 , 0 };
static uint8_t servo_expect[6];
static uint8_t servo_angles[5] = { 90,90,90,90,90};  /* Actual rotation angle value of servo motor */
static uint8_t extended_func_angles[5] = { 80,30,140,110,90}; //Action data
static void servo_control(void); /* Servo control */
```

In action data, the 1st to 5th bits indicate respectively the corresponding angles of No.1 to No.5 servos.

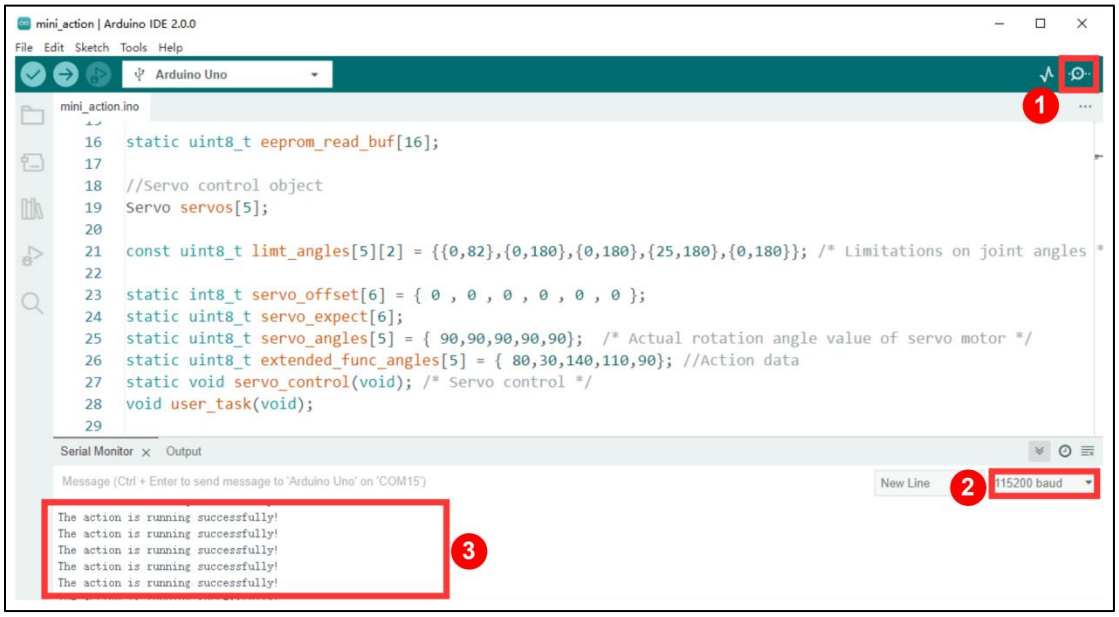
3) Refer to “1.1 Basic Program Download” above to download the program “mini_action.ino” into Arduino UNO.

4) After the download is complete, power the robotic arm on to run the set action.

2.2 Action Running Outcome

Power the robotic arm on and keep the Type-B connected to the computer. Open the serial port monitor to select the baud rate as “115200”.

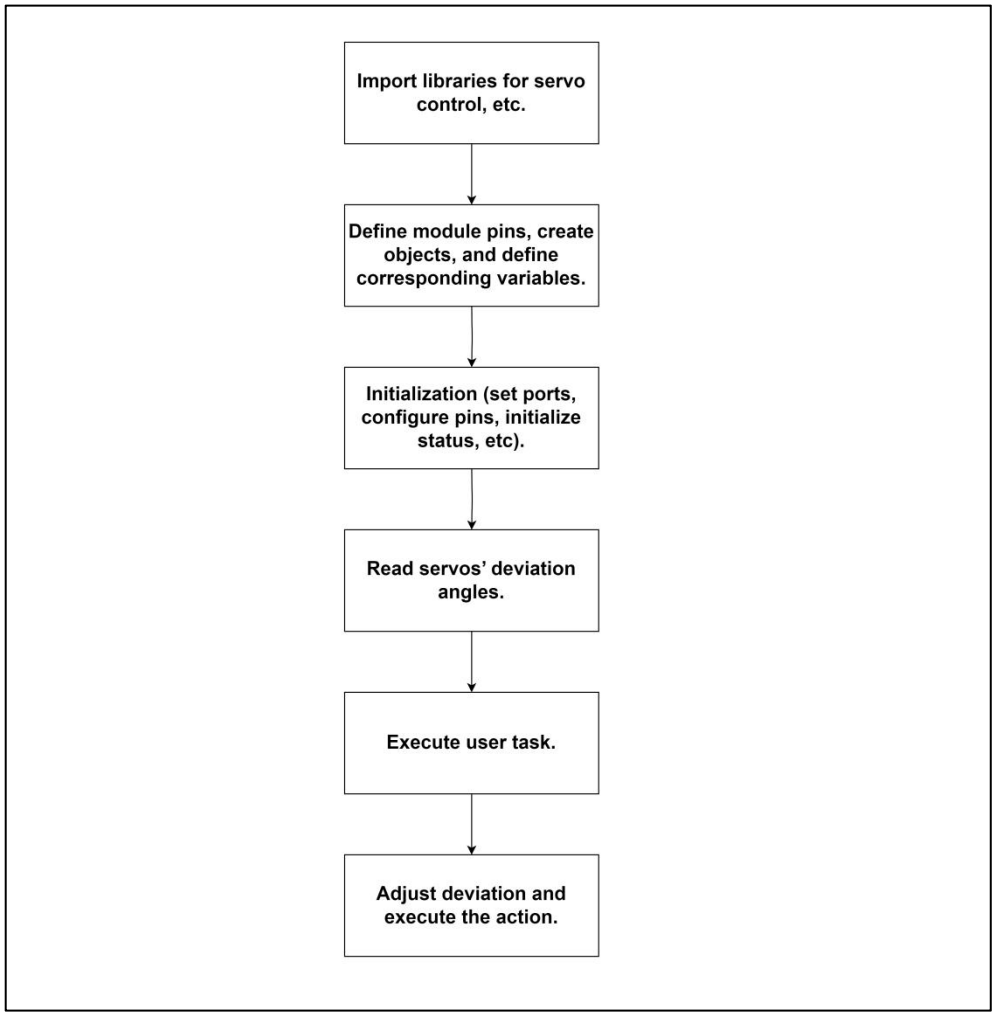
Once the action is completed, the serial port monitor will display “The action is running successfully!”, indicating successful execution of the action.



2.3 Analysis of Action Running Program

The action running program is located in “1.Tutorials\4.Secondary Development Basics and Environment Configuration\4.3 Brief Control Introduction\mini_action\mini_action.ino”.

You can refer to the following flowchart for the implementation logic of the program:



1) Initialize the serial port as “115200”, then the servo pins. Call the error reading function “read_servo_offset()” to read the current deviation of the robotic arm.

```
void setup() {
  // put your setup code here, to run once:
  Serial.begin(115200);
  // Set the serial port's timeout for reading data.
  Serial.setTimeout(500);

  // Assign servos' IO port
  for (int i = 0; i < 5; ++i) {
    servos[i].attach(servoPins[i]);
  }

  //Call error reading function
  read_servo_offset();

  delay(2000);
  Serial.println("start");
}
```

2) In the "user_task()" function, store the codes to be executed for users.

```
void user_task(void)
{
  static uint32_t last_tick = 0;
  if (millis() - last_tick < 100) {
    return;
  }
  last_tick = millis();

  Serial.println("The action is running successfully!");
}
```

3) Next, call "servo_control()" to control servos rotating to corresponding angles for achieving related action. Adjust the deviation of the current robotic arm before running the action.

```
void servo_control(void) {
  static uint32_t last_tick = 0;
  if (millis() - last_tick < 20) {
    return;
  }
  last_tick = millis();

  for (int i = 0; i < 5; ++i) {
    servo_expect[i] = extended_func_angles[i] + servo_offset[i];
    if(servo_angles[i] > servo_expect[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + servo_expect[i] * 0.1;
      if(servo_angles[i] < servo_expect[i])
        servo_angles[i] = servo_expect[i];
    }else if(servo_angles[i] < servo_expect[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + (servo_expect[i] * 0.1 + 1);
      if(servo_angles[i] > servo_expect[i])
        servo_angles[i] = servo_expect[i];
    }

    servo_angles[i] = servo_angles[i] < limt_angles[i][0] ? limt_angles[i][0] : servo_angles[i];
    servo_angles[i] = servo_angles[i] > limt_angles[i][1] ? limt_angles[i][1] : servo_angles[i];
    servos[i].write(i == 0 || i == 5 ? 180 - servo_angles[i] : servo_angles[i]);
  }
}
```

If modifying the rotation speed of the robotic arm's servos, you should ensure that the sum of actual and desired values is "1".

For instance, if you modify the actual value as "0.5", you should change the corresponding desired value to "0.5". After modifying, download the program, and you will see a significant increase in the rotation speed of the robotic arm's servos.